

Prompt Engineering Contest

Use Case:

"Use AI to Identify and Fix a Security Vulnerability in Legacy Code"

Context:

Your team is modernizing a legacy web application built with an older stack (Node.js + Express). You suspect that one of the modules that handles user authentication and input processing is vulnerable to security issues, particularly SQL Injection or Insecure Session Handling.

You're using an AI assistant to help:

- Analyze the **vulnerable code** (which can be shared in the prompt)
- **Identify the specific vulnerability**
- **Suggest and explain a secure fix**
- Generate **test cases** to validate the fix

Your Task (as Prompt Engineers):

Create a **powerful, structured prompt** that guides the AI assistant to perform the following:

1. Identify the **type of vulnerability** in the legacy code
2. Generate a **secure and modern fix** for the issue
3. Explain the fix clearly, including **why it works** and **what the original flaw was**
4. Provide **test cases** to confirm the issue is fixed
5. Output everything in a structured format (e.g., markdown or code-comment block)

Objective:

To produce a **reliable, reproducible prompt** that helps AI consistently return:

- Accurate diagnosis of the vulnerability
- Secure fix that aligns with modern standards
- Concise explanation for peers or reviewers



Time Limit: 15 minutes



Evaluation Criteria:

- Clarity and completeness of the prompt
- Use of structured format and roles
- Application of prompting techniques like **few-shot**, **chain of thought**, or **role prompting**
- Quality of expected AI response (secure, accurate, and usable)

Sample vulnerable legacy code

```
// server.js
const express = require('express');
const mysql = require('mysql');
const bodyParser = require('body-parser');

const app = express();
const port = 3000;

// Middleware
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));

// MySQL connection (replace with your DB credentials)
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '', // Update with your password
  database: 'legacydb' // Ensure this DB exists
});

db.connect(err => {
  if (err) {
    console.error('Database connection failed:', err.stack);
    return;
  }
  console.log('Connected to MySQL');
});

// Vulnerable login route
app.post('/login', (req, res) => {
  const username = req.body.username;
  const password = req.body.password;

  const query = `SELECT * FROM users WHERE username = '${username}' AND password = '${password}'`;

  console.log('Executing query:', query); // For debug

  db.query(query, (err, results) => {
    if (err) {
      console.error('Query error:', err);
      return res.status(500).send('Database error');
    }

    if (results.length > 0) {
      res.send('Login successful');
    } else {
      res.status(401).send('Invalid credentials');
    }
  });
});

// Health check route
app.get('/', (req, res) => {
  res.send('Legacy Express Server is running');
});

app.listen(port, () => {
  console.log(`Server listening at http://localhost:${port}`);
});
```